Virtual Pet Simulator

Software Engineering

by: Shams Saied

Abstract:

This project presents a Virtual Pet Care System that allows users to adopt and interact with pets by managing their needs such as hunger, health, and happiness. The system provides functionalities including user account management, pet interaction, and notifications. UML diagrams including class, sequence, and activity diagrams were designed to model the system structure and behavior.

Introduction

With the increasing use of digital applications for entertainment and simulation, virtual pet systems have become popular for mimicking real-world responsibilities in a simplified environment. These systems allow users to care for pets by managing their needs and maintaining their well-being.

The objective of this project is to design a Virtual Pet Care System that enables users to:

- Create and manage an account
- Adopt a virtual pet
- Interact with the pet through various actions
- Monitor the pet's health and status
- Receive notifications when the pet needs attention

This system demonstrates object-oriented design principles and uses UML diagrams to represent system structure and workflows clearly.

System Overview

The system consists of four main components:

1. Account

Represents the user of the system. It stores user information and manages interactions with the pet.

2. Pet

Represents the virtual pet that the user adopts and interacts with.

3. Status

Represents the internal condition of the pet, such as hunger and health.

4. Notifications

Handles sending alerts to users when certain conditions are met.

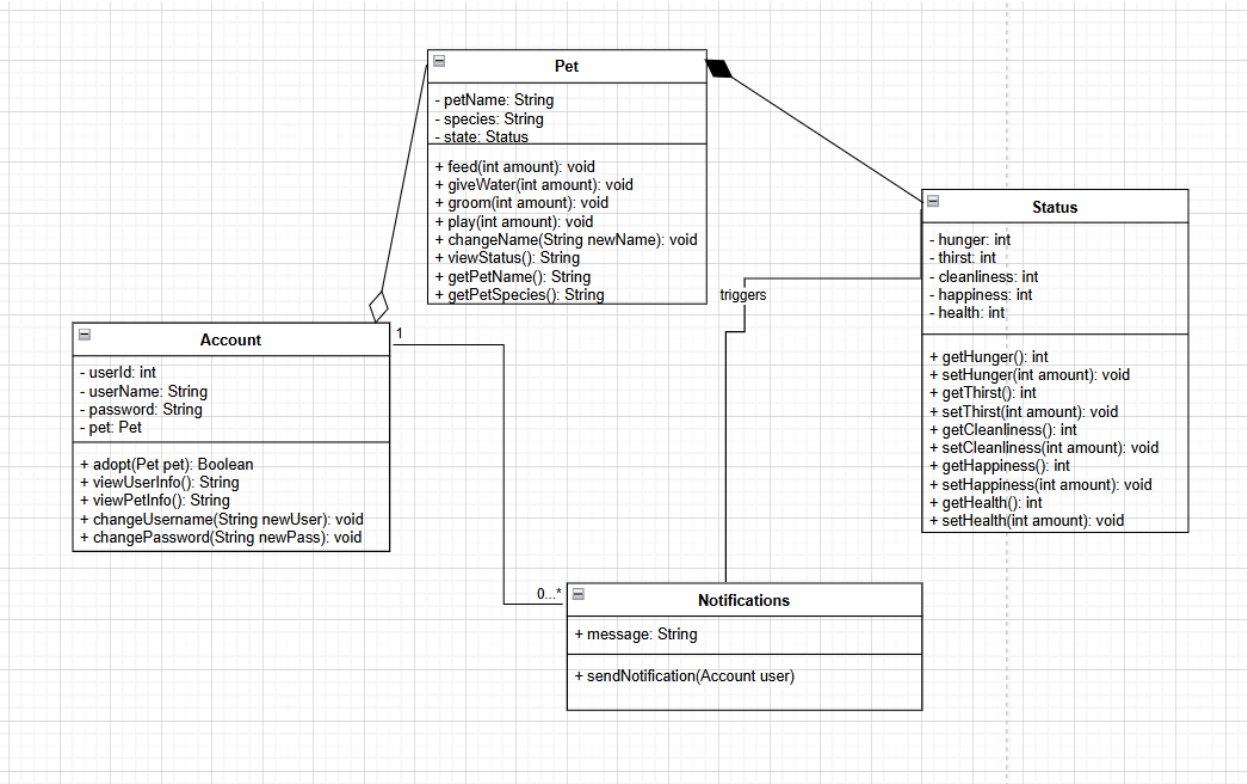
Relationships Between Components

- Each **Account** owns one **Pet**
- Each **Pet** has one **Status**
- **Notifications** are triggered based on changes in the **Status**

UML Diagrams

---

1. Class Diagram



Description

The class diagram represents the static structure of the system.

Classes:

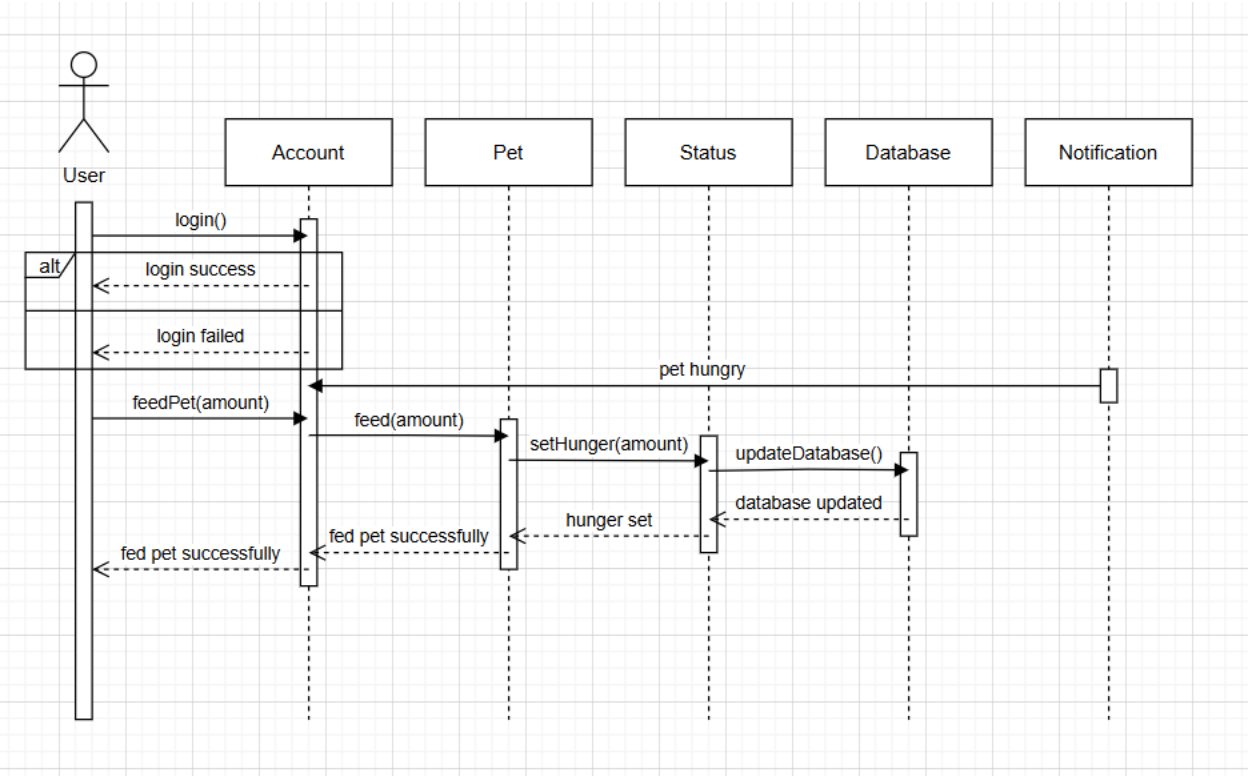
- Account
- Pet
- Status
- Notifications

Relationships:

- Pet and Status: Composition (A pet always has a status; it cannot exist without it)
- Account and Pet: Aggregation (An Account has a Pet)
- Status and Notifications: assosiation(Changes in status trigger notifications)

2. Sequence Diagrams

1. User login and feeding pet



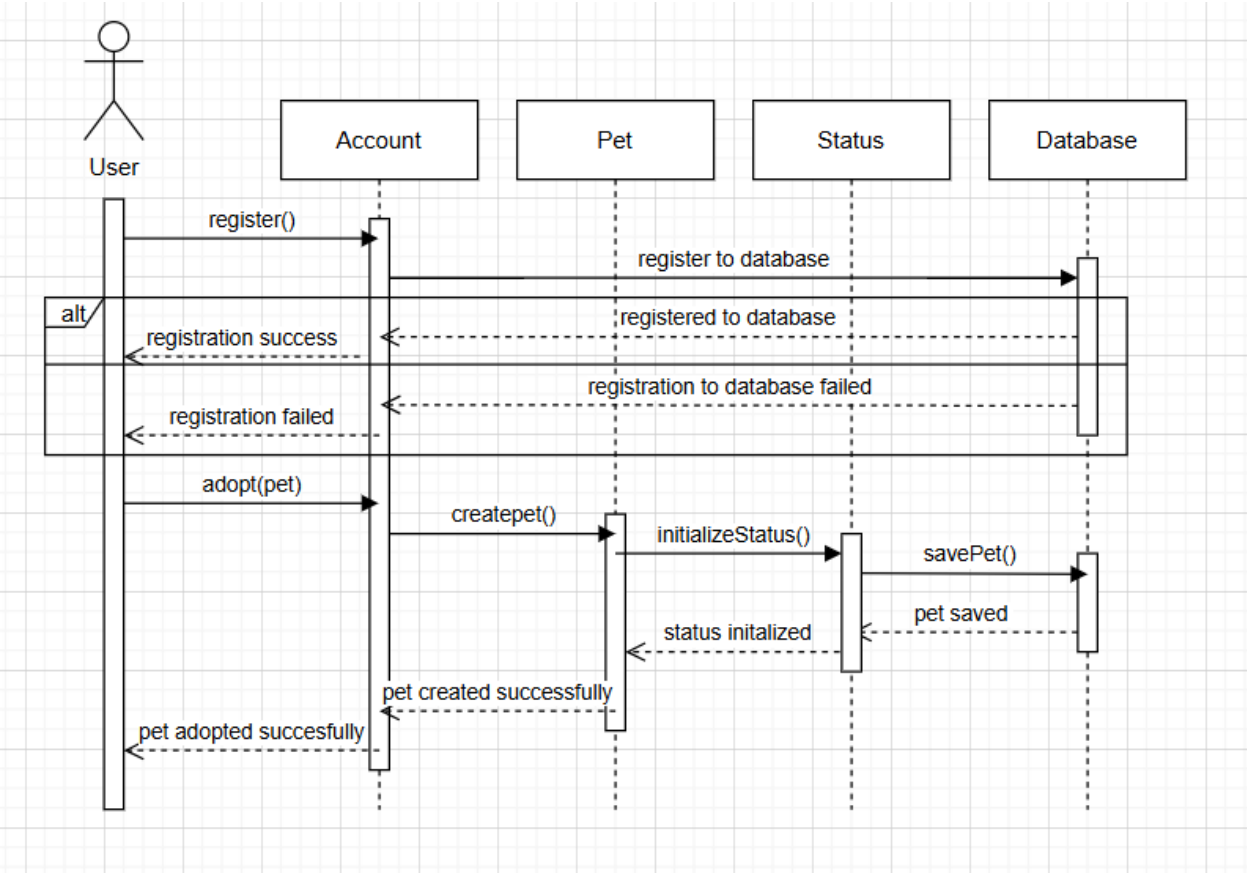
Explanation

- The user initiates login
- Confirmation is returned

- The system notifies use that the pet is hungry
- The user requests to feed pet and provides amount of food
- Pet hunger is updated and so is the stored data for it
- Confirmation is returned

---

2. User registration and Adopting Pet



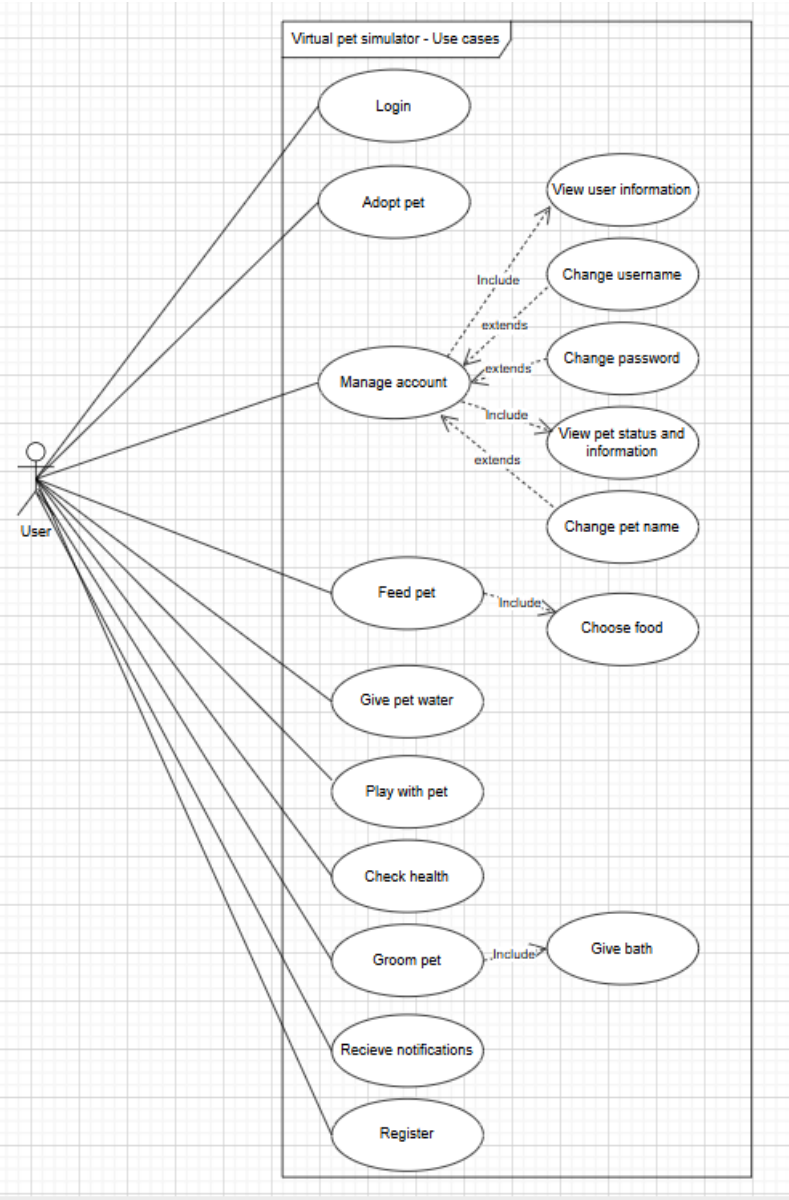
Explanation

- The user initiates registration
- The system creates a new account
- User data is stored
- Confirmation is returned
- The user requests to adopt a pet

- The request is sent to the Account class
- A Pet object is created
- It’s associated status is initialized.
- The pet is linked to the user’s account
- Confirmation is returned

---

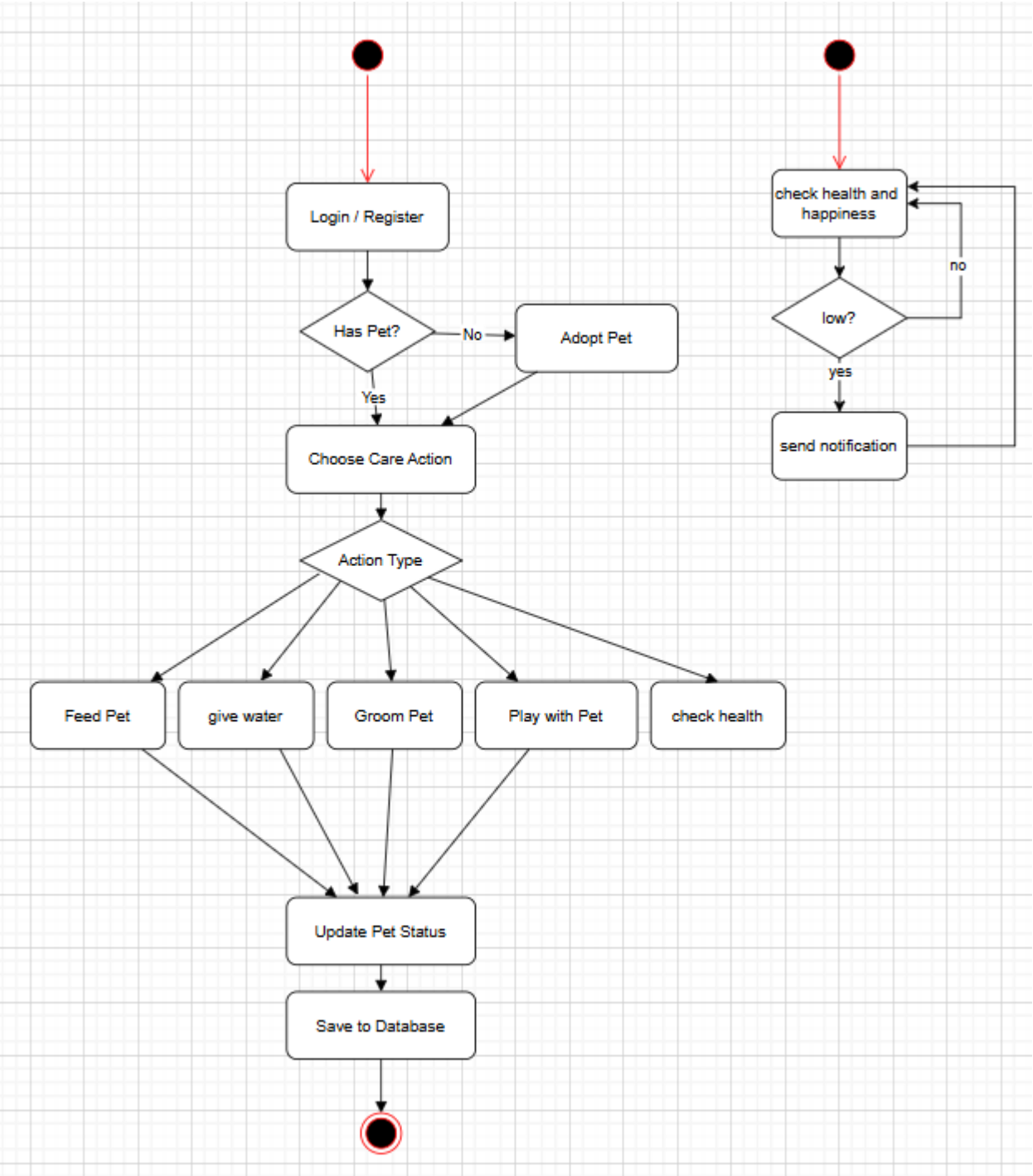
3. Interact with Pet



Explanation

- All the available actions that can be taken by an external user.

5.3 Activity Diagram



Explanation

1. User activity

The activity diagram represents the overall workflow of the system:

1. User starts the system
2. User logs in or registers
3. If no pet exists, adopt a pet
4. User performs actions: feed, give water, groom, play, check health
5. Pet status is updated
6. Change is saved to database

2. Notifications

Notify user when the pet needs something, such as food, play, or a bath.

1. Check pet status
2. If low, notify user
3. Repeat

---

6. Design Decisions

Several design choices were made to improve system quality:

- **Separation of Status Class:**
  The Status class was separated from the Pet class to improve modularity and allow easier updates to pet conditions.
- **Use of Composition:**
  The relationship between Pet and Status ensures that every pet must have a status.
- **Encapsulation:**
  Attributes are accessed using getters and setters to protect data integrity.
- **Dedicated Notification Class:**
  Notifications were separated to allow scalability and future enhancements (e.g., push notifications).
- **Single Responsibility Principle:**
  Each class has a clear and specific responsibility.

---

7. Conclusion

The Virtual Pet Care System demonstrates how UML diagrams can be used to design and visualize a system before implementation. The system successfully models user interaction, pet management, and dynamic status updates. Through this project, important object-oriented design principles such as encapsulation, modularity, and separation of concerns were applied. The UML diagrams provided a clear understanding of both system structure and behavior.

---

8. Future Work

The system can be extended with additional features such as:

- Supporting multiple pets per user
- Adding a graphical user interface (GUI)
- Integrating AI for pet behavior simulation
- Real-time notifications using mobile or web applications
- Saving data in a real database